

CS188 FINAL PROJECT: 2x2 Rubik's Cube Solver

Problem Statement

This project aims to build a LEGO robot capable of solving a 2x2 Rubik's Cube. The robot must:

- Perceive the initial cube state using sensors
 - Compute a sequence of moves to solve the cube
 - Physically manipulate the cube using a robotic gripper or arm to execute the required rotations
-

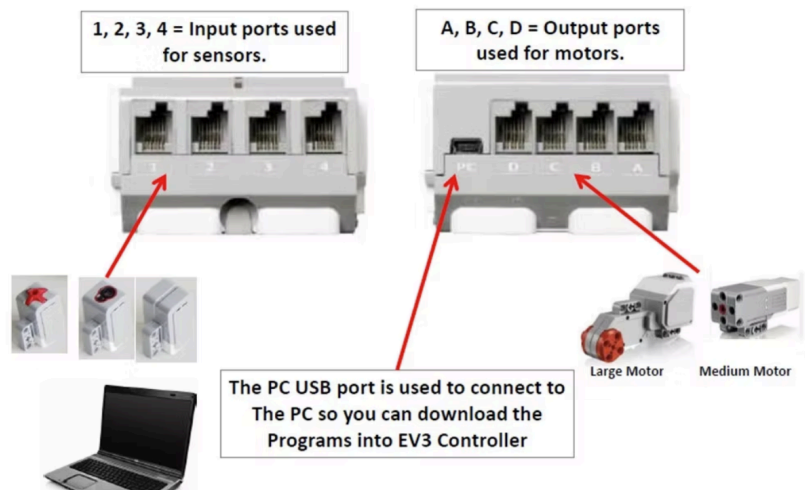
System Design and Methodology

Hardware Setup

We utilized the LEGO Mindstorms EV3 kit as our primary hardware platform, incorporating its onboard sensors, motors, and joints. The EV3 brick includes its own firmware which runs MicroPython, allowing us to interface directly with the robot's hardware using Pybricks. By connecting the EV3 brick to a laptop, we could upload and execute Pybricks programs for full control over the robotic system.

- Example code: hardware initialization using Pybricks (e.g., `Motor(Port.A)`, `ColorSensor(Port.S3)`)
- Code location: `main.py`
- Image:

Ports, Sensors and Motors



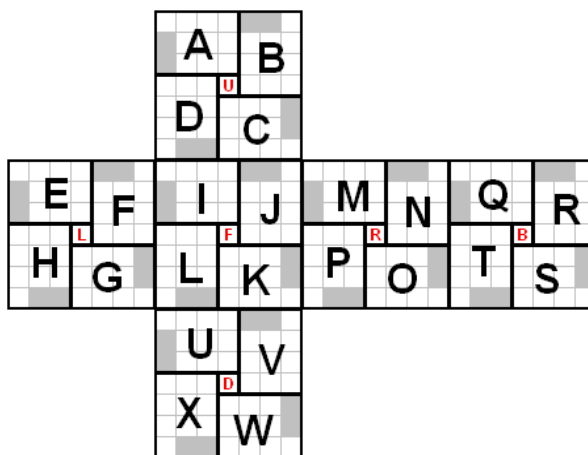
Cube Scanning

To scan the cube, we wrote code to control the arm and sensor mechanism. The cube is flipped systematically while the color sensor reads each sticker. These readings are then stored in an

array using the Speffz letter scheme to represent the cube's state. For example, ["Orange", "Black", ...] represents that orange is the color of the cubie face A, Black is the color of the cubie face B etc.

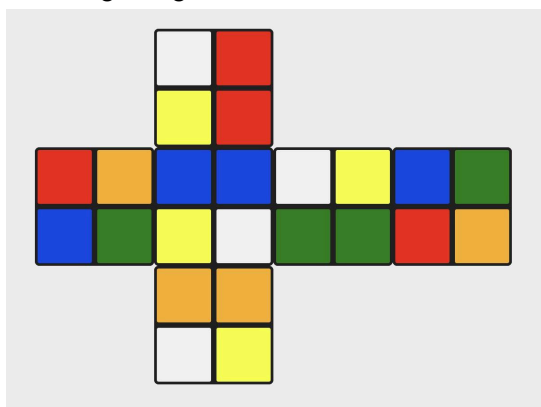
- Example code:


```
def flip():
    # Open arm/reset state
    arm.run_until_stalled(-200, then=Stop.BRAKE, duty_limit=None)
    wait(500)
    ev3.speaker.beep()
    ...
```
- Code location: `main.py`
- Image: Speffz letter scheme



Cubie Color to State Conversion and Encoding

The color array is transformed into a representation of each cubie's permutation and orientation. For example, a permutation and orientation list like [2, 6, 1, 7, 4, 3, 5, 0, 0, 1, 2, 0, 1, 2, 0, 0] indicates the cubie 2 (which is the red, green, white cubie) is in the 0th position (which is the upper back left position) with orientation 0 (white sticker up). Cubie 6 is in the 1st position with orientation 1 (1 clockwise twist until yellow sticker faces up). This state is represented in the following image:



This representation was made more efficient by fixing the permutation and orientation of 1 cubie (known as the DBL piece) so our 16-number state representation which allowed us to uniquely identify cube states became a 14-number state representation (since the orientation and permutation of the DBL piece is no longer needed). The permutation-orientation state is converted into a single integer, which serves as an offset for a table lookup.

- Example code: color array to permutation/orientation conversion
- Code location: `solver.py` (contains state encoding), `cube.py` (contains 16 to 14 state conversion helper)
- Image:

```

/// The 7 pieces, numbered 0 through 6, are as follows (note DBL not included):
/// 0: UBL
/// 1: UBR
/// 2: UFR
/// 3: UFL
/// 4: DFL
/// 5: DFR
/// 6: DBR
///
/// Illustration of the 7 pieces on the cube:
///
/// +---+
/// | 0 1 |
/// | 3 2 |
/// +---+
/// | 4 5 |
/// | 6  |
/// +---+

```

Table Lookup

We used a precomputed binary file (`massive_table.bin`) that contains solutions which are a sequence of human moves for every cube state. This table was generated using Eric Wu's Massive Table Generator: <https://github.com/ericwu17/Massive-Table-Generator/commits/main/> This table generator efficiently calculates the human solution for each state.

- Code location: `solver.py` (searches for human solution in `massive_table.bin`)

Human Solution Moves to Robot Actions

The solution from the table consists of human moves, which had to be translated into robot-executable instructions. The resulting robot instructions are then executed by EV3 motors.

- Code location: `solver.py` (contains human moves to robot executable moves), `cube.py` (contains helper functions to convert human moves to robot executable moves), `main.py` (functions that manipulate robot arms to execute the solution)
-

Evaluation Results

During development, we validated individual components such as motor movements and color detection. For testing, we ran the robot five times on scrambled cubes. It successfully solved the cube in all five trials.

Link to demo: <https://youtu.be/RNKrxE3Xjk0>

Discussion and Reflections

Hardware Issues

Since the Mindstorms EV3 kit has been discontinued, we sourced a second-hand kit from eBay, which was missing several parts. To find an efficient design that minimizes part usage, we adapted David Gilday's LEGO design: <https://mindcuber.com/mindcub3r/mindcub3r.html> with substitutions and structural modifications.

We had to further modify David Gilday's LEGO design by reducing the dimensions of the turntable and flipper arm since his was for a 3x3 cube and we wanted to build a robot solver for a 2x2 cube.

Sensor Challenges

The EV3 color sensor had difficulty distinguishing between yellow and orange, so we painted one face black for contrast. The cube's glossy surface also interfered with detection, which we fixed by adding matte tape.

Optimizations for Table Access

To optimize lookup time, we encoded the cube state into a single integer. We also used Python's `seek()` function to access the correct byte offset in `massive_table.bin`. This was critical because the 22MB binary file was too large to be loaded into the EV3's memory all at once. Seek allowed direct access, reducing lookup overhead significantly.

- Code location: `solver.py`